how you can include that in the order-processing logic implemented in `OrderService`.

The `submitOrder()` method in the `OrderService` class is currently rejecting orders all the time. But not for long. You can now autowire a `BookClient` instance and use the underlying `WebClient` to start a reactive stream to process the book information and create an order. The `map()` operator lets you map a `Book` to an accepted `Order`. If `BookClient` returns an empty result, you can define a rejected `Order` with the `defaultIfEmpty()` operator. Finally, the stream is ended by calling `OrderRepository` to save the order (either as accepted or rejected).

Listing 8.22 Calling `BookClient` to get the book info when ordered

```
package com.polarbookshop.orderservice.order.domain;

import com.polarbookshop.orderservice.book.Book;
import com.polarbookshop.orderservice.book.BookClient;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import org.springframework.stereotype.Service;

@Service
public class OrderService {
  private final BookClient bookClient;
  private final OrderRepository orderRepository;

  public OrderService(
   BookClient bookClient, OrderRepository orderRepository
  ) {
    this.bookClient = bookClient;
    this.orderRepository = orderRepository;
  }

  ...

  public Mono<Order> submitOrder(String isbn, int quantity) {
    return bookClient.getBookByIsbn(isbn)                          ❶
      .map(book -> buildAcceptedOrder(book, quantity))             ❷
      .defaultIfEmpty(                                             ❸
        buildRejectedOrder(isbn, quantity)
```